

Análisis Matemático del Motor DSP de EdgeSDR-Nexus

Respuestas basadas en código fuente verificado

Generado por inspección de código

Septiembre 2026

Contents

1	Qué representa Pxx y sus unidades	3
1.1	Definición	3
1.2	Método por defecto: PFB	3
1.3	Fórmula completa (PFB)	3
1.3.1	Filtro prototipo Kaiser	3
1.3.2	Componentes polifase	3
1.3.3	Procesamiento por bloque	4
1.3.4	Escalado final	4
1.4	Método alternativo: Welch	4
1.5	Conversión a dBm	4
1.6	Unidades reales	4
2	Cómo se normaliza el motor DSP	5
2.1	Normalización PFB (por defecto)	5
2.2	Normalización Welch (alternativa)	5
3	Integración de potencia y uso de Δf	5
3.1	Conversión dBm $\rightarrow$ W/Hz	5
3.2	Integración discreta uniforme	6
3.3	Integración trapezoidal	6
3.4	Ruta especial para TDT/banda ancha Colombia	6
4	¿Es power_dbm absoluto o relativo?	6
5	Cómo funciona Cumple_P	6
5.1	Fórmula	7
5.2	Valores de entrada	7
5.3	Conversión de la licencia	7
6	De dónde sale 1,363 y cómo se calcula la RBW	7
6.1	El valor en el código	7
6.2	Fórmula teórica del ENBW	8
6.3	Verificación numérica	8
6.4	Cómo se usa el ENBW en el código	8
6.5	Cálculo de la RBW efectiva	8

7	Ajuste FM: piloto de 19 kHz $\rightarrow$ ppm	9
7.1	Método 1: Piloto de tono	9
7.2	Método 2: Frecuencias legales	9
7.3	Almacenamiento y aplicación	9
8	Centroide, MER y BER	10
8.1	Centroide	10
8.2	MER (Modulation Error Ratio)	10
8.2.1	Paso 1: Ruido de fondo	10
8.2.2	Paso 2: SNR (Carrier-to-Noise)	10
8.2.3	Paso 3: Métricas espectrales	10
8.2.4	Paso 4: Fórmula del MER	10
8.3	BER (Bit Error Rate)	11
9	Calibración real: qué existe y qué corrige	11
9.1	Calibración de frecuencia (PPM)	11
9.2	Corrección de ganancia por frecuencia	11
9.3	Compensación de desbalance I/Q	11
9.4	Lo que NO existe	12
9.5	Resumen de capas de calibración	12
10	Resumen: PFB vs Welch	12

1 Qué representa Pxx y sus unidades

[Node/C - motor de cómputo espectral]

1.1 Definición

Pxx es la **densidad espectral de potencia (PSD)** calculada mediante el método **PFB (Polyphase Filter Bank)** por defecto, o Welch (configurable), expresada en **dBm relativo a la escala digital del ADC, asumiendo impedancia de 50 Ω** .

Fuente: Edge-Node/rf/libs/psd.h, líneas 179–224.

“Los valores en dBm son relativos a la escala digital del ADC. Para obtener potencia RF absoluta es necesaria una calibración externa.”

1.2 Método por defecto: PFB

El orquestador (`orchestrator.py`, línea 346) y el campaign runner (`campaign_runner.py`, línea 90) envían `method_psd="pfb"` por defecto. El motor C (`parser.c`, línea 53) usa Welch como fallback si el campo no se especifica.

La arquitectura PFB consiste en:

1. Un filtro FIR prototipo $h[n]$ de longitud $L = M \cdot T$ (ventana Kaiser, $\beta = 8,6$).
2. $T = 8$ taps por canal, $M = \text{nperseg}$ canales.
3. El filtro se descompone en T componentes polifase $p_t[m] = h[tM + m]$.
4. Para cada bloque de entrada, se aplica el filtro polifase y luego una FFT de tamaño M .

Fuente: `psd.c`, líneas 677–847 (`execute_pfb_psd`).

1.3 Fórmula completa (PFB)

Sean $x[n]$ las muestras I/Q del ADC ($n \in [-128, 127]$, int8), F_s la frecuencia de muestreo, $M = \text{nperseg}$ el número de canales, $T = 8$ taps por canal, $L = M \cdot T$ la longitud del filtro, y B el número de bloques procesados ($B = \lfloor (N - L)/M \rfloor$, donde N es el total de muestras).

1.3.1 Filtro prototipo Kaiser

$$h[n] = \frac{I_0\left(\beta \sqrt{1 - \left(\frac{2n}{L-1} - 1\right)^2}\right)}{I_0(\beta)}, \quad \beta = 8,6 \quad (1)$$

donde $I_0(x) = \sum_{k=0}^{\infty} \frac{(x^2/4)^k}{(k!)^2}$ es la función de Bessel modificada. $\beta = 8,6$ proporciona ~ 80 dB de atenuación de lóbulos laterales.

Fuente: `psd.c`, líneas 668–674.

1.3.2 Componentes polifase

$$p_t[m] = h[t \cdot M + m], \quad t = 0, \dots, T-1, \quad m = 0, \dots, M-1 \quad (2)$$

1.3.3 Procesamiento por bloque

Para cada bloque b :

$$X_b[m] = \text{FFT}_M \left\{ \sum_{t=0}^{T-1} x[bM + tM + m] \cdot p_t[m] \right\} \quad (3)$$

Se acumula la magnitud al cuadrado:

$$S[m] = \sum_{b=0}^{B-1} |X_b[m]|^2 \quad (4)$$

1.3.4 Escalado final

$$\hat{P}(m) = \frac{S[m]}{B \cdot F_s \cdot M} \quad (5)$$

Fuente: `psd.c`, línea 823:

```
1 double scale = 1.0 / (blocks * fs * M);
```

Nota: A diferencia de Welch, el PFB **no usa factor de energía de ventana** (U). La atenuación de leakage está controlada por el diseño del filtro Kaiser prototipo, no por una normalización adicional.

1.4 Método alternativo: Welch

Cuando `method_psd="welch"`, se usa el estimador clásico de Welch:

$$\hat{P}_{\text{Welch}}(k) = \frac{1}{K} \sum_{i=0}^{K-1} \frac{|X_i(k)|^2}{F_s \cdot L \cdot U} \quad (6)$$

donde $U = \frac{1}{L} \sum_{n=0}^{L-1} |w[n]|^2$ es la energía normalizada de la ventana, K el número de segmentos traslapados, y $L = \text{nperseg}$.

Fuente: `psd.c`, línea 598:

```
1 double scale = 1.0 / (fs * u_norm * k_segments * nperseg);
```

1.5 Conversión a dBm

Ambos métodos convierten el resultado lineal a dBm mediante:

$$P_{\text{dBm}}(k) = 10 \log_{10} \left(\frac{\hat{P}(k)}{50} \times 1000 \right) = 10 \log_{10}(\hat{P}(k)) + 13,01 \quad (7)$$

Fuente: `psd.c`, líneas 319–331 (`convert_to_dbm_inplace`).

1.6 Unidades reales

- La salida cruda del PFB (o Welch) es densidad espectral de potencia.
- Las muestras del ADC son **enteros de 8 bits** $[-128, 127]$, sin unidades de voltaje reales.
- Por lo tanto, **Pxx** es **dBm relativo a la escala digital**, no dBm absoluto en la antena. Para obtener dBm absoluto se necesita calibración externa.

2 Cómo se normaliza el motor DSP

[Node/C - motor de cómputo espectral]

2.1 Normalización PFB (por defecto)

El PFB no aplica factor de ventana separado. La normalización es:

$$\text{scale} = \frac{1}{B \cdot F_s \cdot M} \quad (8)$$

Factor	Valor	Propósito
B	Bloques procesados	Promedio sobre bloques
F_s	Frecuencia de muestreo	Convertir a densidad por Hz
M	Número de canales (nperseg)	Normalizar amplitud de FFT

El filtro Kaiser prototipo ($\beta = 8,6$) controla el leakage espectral mediante su diseño, no mediante una normalización adicional.

2.2 Normalización Welch (alternativa)

Cuando se selecciona Welch, la normalización incluye el factor de energía de ventana:

$$U = \frac{1}{L} \sum_{n=0}^{L-1} |w[n]|^2 \quad (9)$$

$$\text{scale}_{\text{Welch}} = \frac{1}{F_s \cdot U \cdot K \cdot L} \quad (10)$$

Factor	División	Propósito
F_s	Frecuencia de muestreo	Convertir a densidad por Hz
U	Energía normalizada de ventana	Compensar atenuación de ventana
K	Número de segmentos	Promedio de Welch
L	Longitud de segmento (nperseg)	Normalizar amplitud de FFT

3 Integración de potencia y uso de Δf

[Back/Postprocesamiento - power_utils.py, spectral_analysis.py]

3.1 Conversión dBm $\rightarrow$ W/Hz

`SpectrumFrame.to_power_w()` (`spectrum_frame.py`, línea 62):

$$P_{\text{W/Hz}} = 10^{\frac{P_{\text{dBm}} - 30}{10}} \quad (11)$$

Nota: Esta conversión trata las amplitudes como W/Hz, consistente con la salida del motor PSD (densidad espectral).

3.2 Integración discreta uniforme

`channel_power_dbm_uniform_bins()` (`power_utils.py`, línea 118):

$$P_W = \sum_k P_{W/Hz}(k) \cdot \Delta f_{\text{bin}} \quad (12)$$

donde Δf_{bin} se obtiene de `frame.bin_hz` o, si no está disponible, de la mediana de $|\Delta f|$ entre bins vecinos.

```
1 p_w = float(np.sum(p_w_per_hz) * bin_hz)
2 return 10.0 * float(np.log10(p_w / 1e-3))
```

Fuente: `power_utils.py`, líneas 97–131.

3.3 Integración trapezoidal

`measure_channel_power()` (`spectral_analysis.py`, línea 1815):

$$P_W = \int P_{W/Hz}(f) df \approx \text{trapz}(P_{W/Hz}, f) \quad (13)$$

Sí, se usa Δf siempre. La integración es **absoluta** (no por bin), multiplicando la densidad espectral por el ancho de frecuencia correspondiente.

3.4 Ruta especial para TDT/banda ancha Colombia

Para spans ≥ 2 MHz sobre la banda 470–698 MHz, `power_utils.py` línea 112 usa el **nivel medio** en banda en lugar de integrar:

$$P_{\text{dBm}} = 10 \log_{10} \left(\frac{\bar{P}_{W/Hz}}{10^{-3}} \right) \quad (14)$$

4 ¿Es power_dbm absoluto o relativo?

[Back/Postprocesamiento - `processor.py`; Node/C - motor PFB]

`power_dbm` es la **potencia integrada en la banda de la emisión** calculada como:

$$\text{power_dbm} = \text{channel_power_dbm_uniform_bins}(\text{sub-emisión}) \quad (15)$$

Fuente: `processor.py`, línea 902.

Es relativo al ADC. Los valores provienen de la cadena:

1. [Node/C] ADC int8 $\rightarrow$ muestras $x[n] \in [-128, 127]$
2. [Node/C] Motor DSP (PFB por defecto) con normalización por B, F_s, M
3. [Node/C] Conversión a dBm con offset de 50 Ω (`convert_to_dbm_inplace`)
4. [Back] Integración sobre la banda de emisión

No hay corrección por ganancia de LNA/VGA/AMP, pérdida de cable, ni factor de ruido del receptor. El dBm es **histórico/nominal**, no absoluto en la antena.

5 Cómo funciona `Cumple_P`

[Back/Postprocesamiento - `processor.py`; API/Compliance - CSV licencias ANE]

5.1 Fórmula

Cumple_P es una comparación umbral directa:

$$\text{Cumple_P} = \begin{cases} \text{"SI"} & \text{si } P_{\text{medida}} \leq P_{\text{nominal}} \\ \text{"NO"} & \text{si } P_{\text{medida}} > P_{\text{nominal}} \end{cases} \quad (16)$$

Fuente: processor.py, líneas 1612–1617:

```
1 if lic_match == "SI" and p_nominal_dBm is not None and p_medida_dBm is
  not None:
2     try:
3         cumple_p = "SI" if float(p_medida_dBm) <= float(p_nominal_dBm)
4         else "NO"
5     except Exception:
        cumple_p = None
```

5.2 Valores de entrada

- P_{medida} : p_medida_dBm = resultado de channel_power_dbm_uniform_bins() (potencia integrada en la emisión, relativa al ADC). [\[Back\]](#)
- P_{nominal} : p_nominal_dBm = potencia de la licencia proveniente del CSV de la ANE, convertida a dBm por _power_to_dbm() (calibration_io.py, línea 353). [\[API/Compliance\]](#)

5.3 Conversión de la licencia

_power_to_dbm() acepta dBm, dBW, W, mW, kW, μ W y los convierte todos a dBm:

```
1 def _power_to_dbm(p: float, unit: str) -> float:
2     if u in ("DBM", "DB", "DBM."): return float(p)
3     if u in ("DBW", "DBW."): return float(p) + 30.0
4     if u in ("W", "WATTS", "WATT"): return 10.0 * math.log10(p) + 30.0
5     # ... etc
```

Fuente: calibration_io.py, líneas 353–403.

6 De dónde sale 1,363 y cómo se calcula la RBW

[\[Node/C - psd.c, psd.h\]](#)

6.1 El valor en el código

get_window_enbw_factor() (psd.c, línea 336) retorna un valor **hardcodeado**:

```
1 case HAMMING_TYPE: return 1.363;
```

Nota importante: Este factor se usa **exclusivamente** en find_params_psd() para calcular N_{FFT} a partir de la RBW deseada. El PFB **no utiliza** este factor en su normalización; el Welch sí.

Nota: El valor 1,3008 **no existe** en el código fuente. El valor real usado es **1.363**.

6.2 Fórmula teórica del ENBW

El Equivalent Noise Bandwidth de una ventana se define como:

$$\text{ENBW} = N \cdot \frac{\sum_{n=0}^{N-1} |w[n]|^2}{\left(\sum_{n=0}^{N-1} w[n]\right)^2} \quad (17)$$

Para la ventana Hamming estándar $w[n] = 0,54 - 0,46 \cos(2\pi n/M)$ con N grande:

$$\frac{1}{N} \sum w[n]^2 \rightarrow 0,54^2 + \frac{0,46^2}{2} = 0,2916 + 0,1058 = 0,3974 \quad (18)$$

$$\left(\frac{1}{N} \sum w[n]\right)^2 \rightarrow 0,54^2 = 0,2916 \quad (19)$$

$$\text{ENBW} = \frac{0,3974}{0,2916} \approx 1,3628 \approx 1,363 \quad (20)$$

6.3 Verificación numérica

Se verificó computando los coeficientes reales de NumPy/SciPy:

```
numpy hamming (N=1024):          ENBW = 1.363784
scipy hamming sym=True (N=1024): ENBW = 1.363784
Analytical (a0=0.54, a1=0.46):  ENBW = 1.362826
```

El valor 1,363 en el código es una aproximación redondeada a 3 decimales del ENBW teórico de Hamming, consistente con la implementación de la ventana $w[n] = 0,54 - 0,46 \cos(2\pi n/M)$.

6.4 Cómo se usa el ENBW en el código

El factor ENBW se usa en `find_params_psd()` (`psd.c`, líneas 254–304) para calcular N_{FFT} :

$$N_{\text{FFT}} = 2^{\lceil \log_2(\text{ENBW} \times F_s / \text{RBW}_{\text{deseada}}) \rceil} \quad (21)$$

```
1 double enbw_factor = get_window_enbw_factor(desired.window_type);
2 double required_nperseg_val = enbw_factor * desired.sample_rate /
   safe_rbw;
3 int exponent = (int)ceil(log2(required_nperseg_val));
4 psd_cfg->nperseg = (int)pow(2, exponent);
```

6.5 Cálculo de la RBW efectiva

La RBW efectiva es:

$$\text{RBW}_{\text{efectiva}} = \text{ENBW} \times \frac{F_s}{N_{\text{FFT}}} \quad (22)$$

Se **distingue** de $\Delta f = F_s/N_{\text{FFT}}$ (espaciado de bins). Para Hamming:

$$\text{RBW} = 1,363 \times \frac{F_s}{N_{\text{FFT}}} > \Delta f \quad (23)$$

7 Ajuste FM: piloto de 19 kHz → ppm

[Node/Tools - `kal_sync_pilot_tone.py`, `kal_sync_legal_FM.py`; Node/C - aplicación de corrección]

7.1 Método 1: Piloto de tono

1. Captura I/Q a $F_c = 98$ MHz, $F_s = 20$ MHz.
2. Desmodula FM: `audio = angle(iq[1:] * conj(iq[:-1]))`.
3. Calcula Welch del audio (decimado a 200 kHz).
4. Busca picos en la banda 18–20 kHz, identifica el piloto estéreo.
5. Refina la frecuencia del piloto con interpolación espectral.
6. Calcula el offset fino Δf buscando simetría espectral:

$$\Delta f^* = \arg \min_d \frac{1}{N} \sum_u \frac{(\hat{P}(d-u) - \hat{P}(d+u))^2}{\hat{P}(d-u) + \hat{P}(d+u)} \quad (24)$$

7. Convierte a ppm:

$$\text{ppm} = \frac{\Delta f^*}{f_{\text{estación}}} \times 10^6 \quad (25)$$

Fuente: `kal_sync_pilot_tone.py`, líneas 112–120.

7.2 Método 2: Frecuencias legales

1. Descarga la base de datos de frecuencias legales de la ANE (CSV con lat/lng). [DB]
2. Selecciona emisoras FM dentro de un radio de cobertura.
3. Detecta picos en el espectro PSD y los matchea con las frecuencias legales:

$$\text{ppm}_i = \frac{f_{\text{medida},i} - f_{\text{legal},i}}{f_{\text{legal},i}} \times 10^6 \quad (26)$$

4. Usa **descenso de gradiente** para minimizar el error ppm residual:

$$c_{n+1} = c_n - \eta \cdot \text{error}(c_n), \quad \eta = 0,8 \quad (27)$$

5. Convergencia: early stopping si $|\text{error}| < 1$ ppm o paciencia agotada.

Fuente: `kal_sync_legal_FM.py`, líneas 128–189.

7.3 Almacenamiento y aplicación

El valor ppm se guarda en `/dev/shm/persistent.json` como `ppm_error` [Node/SHM] y se aplica en el motor C [Node/C]:

$$f_{\text{corregida}} = f_{\text{nominal}} \times \left(1 + \frac{\text{ppm}}{10^6}\right) \quad (28)$$

Fuente: `psd.c`, líneas 286–287.

8 Centroide, MER y BER

[Back/Postprocesamiento - spectral_analysis.py]

8.1 Centroide

No existe una función explícita de **centroide espectral** en el código. El “centro” de una emisión se calcula como:

$$f_{\text{centro}} = \frac{f_{\text{inicio}} + f_{\text{fin}}}{2} \quad \text{o} \quad f_{\text{centro}} = f[k_{\text{pico}}] \quad (29)$$

El cálculo de OBW (`_obw_99_limits_in_segment`, `spectral_analysis.py`) encuentra los límites de frecuencia que contienen el 99% de la potencia y reporta:

$$f_{\text{OBW}} = \frac{f_{i_0} + f_{i_1}}{2} \quad (30)$$

8.2 MER (Modulation Error Ratio)

El MER es una **estimación heurística basada en la forma del espectro**, no una medición de constelación.

Fuente: `spectral_analysis.py`, función `estimate_mer_ber_from_spectrum_tdt()`, líneas 1986–2058.

8.2.1 Paso 1: Ruido de fondo

Se usa el percentil 50 de la banda guardia inferior, con estimador robusto MAD:

$$P_{\text{NF}} = \text{mediana}(y_{\text{guardia, baja}}) + 0,15 \times 1,4826 \times \text{MAD}(y_{\text{guardia, baja}}) \quad (31)$$

8.2.2 Paso 2: SNR (Carrier-to-Noise)

$$C/N = 10 \log_{10} \left(\frac{\max(P_{\text{señal}} - P_{\text{NF}}, 0)}{P_{\text{NF}}} \right) \quad (32)$$

8.2.3 Paso 3: Métricas espectrales

$$\text{exceso}_{\text{med}} = \text{mediana}(y_{\text{canal}}) - P_{\text{NF}} \quad (33)$$

$$\text{exceso}_{\text{p90}} = P_{90}(y_{\text{canal}}) - P_{\text{NF}} \quad (34)$$

$$\text{ripple} = \text{desv. estándar}(y_{\text{canal}} - \text{mediana}(y_{\text{canal}})) \quad (35)$$

$$\text{notch} = \max(0, \text{mediana}(y_{\text{canal}}) - P_{10}(y_{\text{canal}})) - 1,5 \quad (36)$$

8.2.4 Paso 4: Fórmula del MER

$$\text{MER} = 0,55 \cdot \frac{C}{N} + 0,30 \cdot \text{exceso}_{\text{p90}} + 0,15 \cdot \text{exceso}_{\text{med}} - 0,35 \cdot \text{ripple} - 0,20 \cdot \text{notch}$$

(37)

con $\text{MER} \in [0, 40]$ dB (recortado).

Nota: Esta es una regresión heurística ponderada, **no** una medición de MER de constelación. Es válida solo para TDT (UHF 470–698 MHz, 64-QAM).

8.3 BER (Bit Error Rate)

El BER se estima **a partir del MER** usando la fórmula teórica de M-QAM en AWGN:

$$\text{BER} = \frac{4}{k} \left(1 - \frac{1}{\sqrt{M}}\right) Q\left(\sqrt{\frac{3k \cdot \text{SNR}_{\text{lin}}}{M-1}}\right) \quad (38)$$

donde $k = \log_2 M$, M es el orden de la modulación, $\text{SNR}_{\text{lin}} = 10^{\text{MER}/10}$, y $Q(x) = \frac{1}{2}\text{erfc}(x/\sqrt{2})$.

Fuente: `spectral_analysis.py`, líneas 2053–2058.

Conclusión: MER y BER **no** provienen de demodulación real ni de extracción de constelación. Son estimaciones basadas en la forma del espectro y el modelo teórico de M-QAM en AWGN.

9 Calibración real: qué existe y qué corrige

[[Node/Tools](#) - calibración; [Node/C](#) - aplicación de corrección; [Back](#) - corrección de ganancia]

9.1 Calibración de frecuencia (PPM)

- **Qué hace:** Corrige el error de frecuencia del oscilador local del HackRF. [[Node/Tools](#)]
- **Método:** Usa emisoras FM legales como referencia (`kal_sync_legal_FM.py`) o el piloto estéreo de 19 kHz (`kal_sync_pilot_tone.py`).
- **Almacenamiento:** `ppm_error` en `/dev/shm/persistent.json`. [[Node/SHM](#)]
- **Aplicación:** $f_{\text{corregida}} = f_{\text{nominal}} \cdot (1 + \text{ppm}/10^6)$ en el motor C. [[Node/C](#)]

9.2 Corrección de ganancia por frecuencia

- **Qué hace:** Aplica una corrección aditiva en dB interpolada por frecuencia. [[Back](#)]
- **Archivo:** CSV con columnas “Frecuencia (MHz)” y “Error (dB)”, cargado vía `--corr-csv`.
- **Fórmula:**

$$A_{\text{corregida}}(f) = A(f) + \text{interp}(f, \text{CSV}) \quad (39)$$

- **Fuente:** `processor.py`, líneas 155–170 (`apply_gain_correction`).

9.3 Compensación de desbalance I/Q

- **Qué hace:** Corrección ciega de 3 etapas: eliminación DC $\rightarrow$ balance de ganancia $\rightarrow$ decorrelación de fase. [[Node/C](#)]
- **Fuente:** `psd.c`, líneas 61–194 (`iq_compensation`).

9.4 Lo que NO existe

- No hay calibración absoluta de potencia del ADC. El comentario en `psd.h` lo confirma explícitamente.
- No hay calibración de factor de ruido (NF) del receptor.
- No hay calibración de ganancia de LNA/VGA/AMP por frecuencia.
- No hay corrección de pérdidas de cable/antena.
- No hay archivo de calibración por sensor individual.

9.5 Resumen de capas de calibración

Capa	Corrige	Ubicación	Método
PPM (frecuencia)	Error del oscilador local	Node/Tools → Node/C	FM legal / piloto 19 kHz
Ganancia (relativa)	Respuesta en freq. relativa	Back	CSV <code>--corr-csv</code>
I/Q imbalance	Desbalance ganancia/fase	Node/C	Blind correction
Absoluta (potencia)	No implementada	—	—
NF / LNA / VGA	No implementada	—	—
Cable / antena	No implementada	—	—

10 Resumen: PFB vs Welch

[Node/C - ambos algoritmos en `psd.c`]

Aspecto	PFB (defecto)	Welch (alternativa)
Método	Banco de filtros polifase	Promediado de periodogramas
Filtro prototipo	Kaiser ($\beta = 8,6$)	Ventana configurable (Hamming)
Taps/canales	$T = 8, M = \text{nperseg}$	$L = \text{nperseg}$ segmentos
Escala	$1/(B \cdot F_s \cdot M)$	$1/(F_s \cdot U \cdot K \cdot L)$
Factor U	No usado	Sí (energía de ventana)
ENBW para RBW	No aplica directamente	Sí: $\text{ENBW} \times F_s / N_{\text{FFT}}$
Selectividad	~ 80 dB (Kaiser)	Depende de ventana elegida
Complejidad	$O(L + M \log M)$ por bloque	$O(L \log L)$ por segmento

Referencias de archivos

Archivo	Ubicación	Contenido
<code>rf/libs/psd.c</code>	Node/C	Motor PFB/Welch, ventanas, ENBW, dBm
<code>rf/libs/psd.h</code>	Node/C	Definiciones, tipos, constantes
<code>orchestrator.py</code>	Node	Configuración PFB por defecto
<code>spectrum_frame.py</code>	Back	SpectrumFrame, <code>to_power_w()</code>
<code>power_utils.py</code>	Back	Integración de potencia
<code>spectral_analysis.py</code>	Back	MER/BER, OBW, emisiones
<code>processor.py</code>	Back	Cumple_P, pipeline completo
<code>calibration_io.py</code>	Back/API	Conversión de potencia, licencias ANE
<code>kal_sync_pilot_tone.py</code>	Node/Tools	Calibración por piloto FM
<code>kal_sync_legal_FM.py</code>	Node/Tools	Calibración por FM legal